

PROYECTO PROGRAMACION 1 - ASTROCADE

Joseph David Gomez Argote, Jonathan Parra Landinez, Nicolas Solarte
Moncada



Universidad Libre
Facultad de Ingeniería
Ingeniero: Andres Felipe Castaño Rubiano

Bogota D.C, Colombia
28 de Septiembre de 2026

Resumen

El presente proyecto documenta el diseño y desarrollo de "**AstroCade**", un sistema desarrollado para simular la gestión de un arcade. El programa permitirá registrar jugadores mediante tarjetas, administrar el saldo disponible para jugar y almacenar los tickets obtenidos en las diferentes máquinas. Cada juego contará con un sistema de puntuación (puntuación) y podrá entregar una cantidad determinada de tickets dependiendo del desempeño del jugador.

Palabras clave: Simulación, Arcade, Tickets, Puntuación

Tabla de Contenidos

1. Introducción.....	4
1.1 Propósito del proyecto.....	4
1.2 Alcance.....	6
1.3 Personal involucrado.....	6
1.4 Objetivo General.....	7
1.4.1 Objetivos específicos.....	8
2. Requerimientos del Sistema.....	9
2.1 Requerimientos Funcionales (RF).....	9
2.2 Requerimientos No Funcionales (RNF).....	10
3. Modelado y Arquitectura del Sistema.....	10
3.1 Descripción del Dominio de Clases.....	10
3.2 Relaciones entre Entidades y Diagrama de Clases UML.....	13
ENTREGA 2do CORTE - FASE 2.....	14
1.1 Propósito del proyecto (Actualización).....	14
1.2 Alcance (Actualización).....	14
2. Requerimientos del Sistema.....	15
3. Modelado y Arquitectura del Sistema.....	16
3.1 Descripción del Dominio de Clases (Reestructuración total).....	16
3.2 Relaciones entre Entidades y Diagrama de Clases UML.....	17
3.3 Diagrama de Clases UML.....	18
3.4 Interfaz Gráfica.....	19

Índice de tablas

Tabla 1. Personal Parra. J. Recurso Autoría propia.....	4
Tabla 2. Personal Gomez. J Recurso Autoría propia.....	4
Tabla 3. Personal Solarte. N Recurso Autoría propia.....	5

Link Github:

<https://github.com/Jonathan-Parra1013/AstroCade>

1. Introducción

Actualmente, los centros de entretenimiento tipo Arcade , han evolucionado reemplazando las fichas y dinero por sistemas digitales que utilizan tarjetas recargables. Estos sistemas permitieron a los usuarios pagar por los juegos, acumular puntos o tickets y canjearlos por premios. AstroCade nace como un proyecto académico que busca simular el funcionamiento básico de un Arcade, teniendo en cuenta elementos como los usuarios, las tarjetas, las máquinas de Juegos y los centros de premios. Para desarrollar esta simulación, se utiliza programación orientada a objetos (POO) en Java, aplicando conceptos como la creación y organización de objetos, el manejo de sus datos y las acciones que pueden realizar cada uno dentro del sistema.

1.1 Propósito del proyecto

El propósito general de este proyecto es construir una solución de software orientada a objetos que simula con precisión el ciclo de vida operativo de un sistema de Arcade. La aplicación permite gestionar el flujo de dinero, saldo virtual, consumo por jugada, asignación de tickets de recompensa y reducción de inventario por canje de premios, garantizando la integridad de la información en todo momento.

De manera específica, el proyecto persigue los siguientes objetivos técnicos y académicos:

- **Demostración de Fundamentos de POO:** Demostrar la correcta separación entre la definición de la clase (plantilla) y la existencia de objetos (instancias en memoria), evidenciando que múltiples entidades de la misma clase conservan estados totalmente independientes entre sí.
- **Garantía de Integridad y Encapsulamiento:** Proteger el estado interno de las entidades (saldos, tickets acumulados) mediante atributos privados e implementar métodos modificadores (*setters*) y constructores parametrizados que incluyan validaciones lógicas (evitando transacciones con montos negativos, accesos sin saldo suficiente).
- **Modelado de Comportamiento del Dominio:** Implementar métodos que no se limiten al acceso de datos (*getters/setters*), sino que ejecuten lógica de negocio real del sistema (ej. el cobro de saldo y acreditación de tickets al jugar en una máquina, o el descuento de tickets al redimir un premio).
- **Arquitectura Escalable:** Establecer una base de código limpia, modular y coherente que cumpla estrictamente con las convenciones de desarrollo en Java, de modo que el sistema pueda escalar progresivamente en iteraciones futuras hacia conceptos avanzados como herencia, colecciones dinámicas de datos y persistencia.

1.2 Alcance

El sistema tiene como alcance el diseño e implementación de un prototipo funcional de un simulador de un arcade, desarrollada a partir de Programación Orientada a Objetos. A través de una arquitectura basada en clases interconectadas que representan los elementos del sistema de manera simulada, el contexto de un arcade. A través de estas clases, el sistema define y procesa los atributos necesarios para almacenar el estado de cada elemento y sus métodos correspondientes. Asimismo el proyecto comprende la integración de múltiples objetos en tiempo de ejecución, para dar vida a cada dinámica simulada.

1.3 Personal involucrado

Nombre	Jonathan Parra Landinez
Rol	Programador
Categoría Profinal	Estudiante de ingeniería de sistemas
Responsabilidad	Análisis de información, diseño y programación
Información de contacto	jonathan-parral@unilibre.edu.co

Tabla 1. Personal Parra. J. Recurso Autoría propia

Nombre	Joseph David Gomez Argote
Rol	Programador
Categoría Profinal	Estudiante de ingeniería de sistemas
Responsabilidad	Análisis de información, diseño y programación
Información de contacto	josephd-gomeza@unilibre.edu.co

Tabla 2. Personal Gomez. J Recurso Autoría propia

Nombre	Nicolas Solarte Moncada
Rol	Programador
Categoría Profinal	Estudiante de ingeniería de sistemas
Responsabilidad	Análisis de información, diseño y programación
Información de contacto	nicolas-solartem@unilibre.edu.co

Tabla 3. Personal Solarte. N Recurso Autoría propia

1.4 Objetivo General

Diseñar e implementar un sistema que simule el funcionamiento de un área recreativa tipo arcade, utilizando los principios fundamentales de la programación orientada a objetos en java, con el propósito de gestionar de manera organizada los usuarios, tarjetas, saldos, máquinas recreativas, tickets y canje de premios, garantizando la integridad y correcta interacción entre los diferentes objetos del sistema.

1.4.1 Objetivos específicos

- Modelar el dominio del sistema AstroCade mediante la identificación y definición de al menos cuatro clases principales (Usuario, Tarjeta, MaquinaArcade y Premios), definiendo sus atributos de instancia y métodos de comportamiento según los requerimientos operativos de un centro de entretenimiento.
- Garantizar el encapsulamiento y la integridad de los datos declarando todos los atributos de instancia bajo visibilidad privada e implementando métodos de acceso (getters/setters) y constructores parametrizados con validaciones lógicas que impidan operaciones inválidas (como transacciones con saldos negativos, consumos sin saldo suficiente).
- Implementar la lógica de interacción de objetos en Java, desarrollando métodos de comportamiento que permitan simular la carga de saldo en tarjetas, el descuento de crédito por cada jugada, la asignación de tickets según el puntaje (puntuación) obtenido y la redención de premios.

- Desarrollar una clase principal de pruebas (Main) que instancie múltiples objetos independientes por cada clase, demostrando mediante salidas formateadas por consola el mantenimiento de estados individuales en memoria y el correcto funcionamiento del ciclo de simulación del arcade

2. Requerimientos del Sistema

Esta sección formaliza las capacidades que debe tener AstroCade.

2.1 Requerimientos Funcionales (RF)

RF01 - Registro e Inicialización	El sistema debe permitir la creación de usuarios y la asignación de tarjetas recargables (Tarjeta) parametrizadas con un número identificador y saldo inicial.
RF02 - Gestión de Saldo y Carga	El sistema debe permitir la recarga de dinero en la tarjeta del usuario, validando que el monto ingresado sea superior a cero.
RF03 - Ejecución de Juegos y Consumo	El sistema debe permitir a una tarjeta interactuar con una MaquinaArcade, descontando el costo de la jugada del saldo disponible y asignando una cantidad de tickets ganados con base en el puntaje (puntuación).
RF04 - Canje de Premios e Inventario	El sistema debe permitir la redención de un premio utilizando los tickets acumulados en la tarjeta.
RF05 - Consulta de Estado	El sistema debe permitir consultar en todo momento el estado actual de cada tarjeta (saldo y tickets acumulados), máquina e inventario de Premios.
RF06 - Demostración de Independencia	La clase principal debe demostrar la ejecución paralela e independiente de al menos tres instancias distintas por cada clase del dominio.
RF07 - Prestar/Transferir Tarjetas	El sistema debe permitir que un objeto de la clase Usuario transfiera la posesión de su objeto Tarjeta a otro objeto Usuario, de modo que el nuevo portador pueda recargar saldo, consumir crédito en las máquinas y acumular tickets en la misma tarjeta.

2.2 Requerimientos No Funcionales (RNF)

RNF01 - Lenguaje y Entorno	El sistema debe estar desarrollado completamente en Java (versión 17 o superior) y compilar sin errores.
RNF02 - Encapsulamiento Estricto	Todos los atributos de las clases deben declararse bajo visibilidad privada (private) y ser manipulados únicamente a través de métodos de acceso y modificación (getters / setters) con validaciones.
RNF03 - Legibilidad y Convenciones	El código debe cumplir con las convenciones estándar de nomenclatura Java (clases en PascalCase, atributos y métodos en camelCase).

3. Modelado y Arquitectura del Sistema

El modelado del sistema **AstroCade** adopta un enfoque orientado a objetos para abstraer los elementos clave de un centro de entretenimiento digital. La arquitectura garantiza la modularidad, la independencia de estados en memoria y la reusabilidad mediante una clara división de responsabilidades entre sus clases componentes.

3.1 Descripción del Dominio de Clases

Para este prototipo, el sistema se compone de cuatro clases principales interconectadas que representan el flujo operativo del arcade:

1. Clase Usuario

Representa al cliente o persona dentro del área recreativa.

- **Responsabilidad:** Administra la identidad del jugador y mantiene la posesión de portabilidad temporal de una tarjeta de juego.
- **Atributos (private):** nombre (String), cedula (String) y tarjeta (Tarjeta).
- **Dinámica de Portabilidad:** El atributo tarjeta almacena una referencia a un objeto de tipo Tarjeta. Si el usuario no posee una tarjeta en determinado momento, este atributo toma un valor null. Esto permite que una misma tarjeta pueda ser prestada, devuelta o transferida entre diferentes objetos Usuario, reflejando la realidad operativa del dominio.

2. Clase Tarjeta

Constituye la entidad autónoma de transacciones económicas y de recompensa.

- **Responsabilidad:** Custodiar e incrementar/decrementar el saldo de crédito disponible y el balance acumulado de tickets de recompensa de forma independiente a la persona que la porte en consola.
- **Atributos (private):** numeroTarjeta (String), saldo (double) y tickets (int).
- **Comportamiento:** Incluye la lógica de validación para recargas de dinero, cobro de jugadas y descuento de tickets tras un canje.

3. Clase MaquinaArcade

Representa las diferentes atracciones físicas del arcade de juegos (ej. Mario Bros, Pacman, Tetris).

- **Responsabilidad:** Simular la interacción de juego, evaluar la disponibilidad de saldo de la tarjeta insertada, descontar el valor de la jugada y otorgar tickets con base en el puntaje (*puntuacion*) registrado.
- **Atributos (private):** nombre (String), precioPartida (double) y tickets(int)

4. Clase Premios

Representa los artículos exhibidos en el centro de redención o mostrador de canje (además de que en una futura versión se tiene pensado agregar un atributo stock para controlar la cantidad de premios que hay disponibles) .

- **Responsabilidad:** Controlar la disponibilidad de inventario en tiempo real y validar el costo en tickets requerido para completar la entrega del bien.
- **Atributos (private):** nombre (String), costoTickets (int).

3.2 Relaciones entre Entidades y Diagrama de Clases UML

La arquitectura de **AstroCade** basa su flexibilidad en las siguientes relaciones del modelado orientado a objetos:

1. **Asociación (Portabilidad Usuario – Tarjeta):** Existe una relación de asociación unidireccional y opcional (multiplicidad 0..1 a 1). Un Usuario puede o no tener asignada una Tarjeta. Esta desvinculación estricta permite que un objeto Usuario A transfiera la referencia de su tarjeta a un objeto Usuario B, permitiendo que este último juegue y acumule tickets en la tarjeta del primero.
2. **Dependencia de Uso (MaquinaArcade – Tarjeta):** Las máquinas no almacenan usuarios ni tarjetas permanentemente; únicamente reciben la referencia de un objeto Tarjeta como parámetro dentro del método de comportamiento jugar(), modificando directamente el saldo y tickets de dicha tarjeta.
3. **Dependencia de Transacción (Premios – Tarjeta):** La clase Premios interactúa con el objeto Tarjeta durante el método canjear(), verificando si el balance de tickets alcanza para la transacción.

ENTREGA 2do CORTE - FASE 2

1.1 Propósito del proyecto (Actualización)

Para este segundo corte, la arquitectura evoluciona incorporando conceptos más avanzados vistos en clase de Programación Orientada a Objetos (POO) como clases abstractas, herencia, polimorfismo, interfaces y interfaz gráfica. Se introducen jerarquías de usuario con control de acceso por contraseña, tarjetas con límites y descuentos diferenciados, gestor dinámico de eventos temporales (como Black Friday y validación automática de cumpleaños), además de diferentes tipos de máquinas en el arcade con unas dando tickets y otras dando premios directamente.

1.2 Alcance (Actualización)

El sistema tiene como alcance el diseño e implementación de un prototipo funcional de un arcade desarrollado bajo los temas aprendidos en clase Java. A diferencia de la versión anterior, este alcance comprende la gestión interactiva mediante un menú por medio de una interfaz gráfica que permite: el registro dinámico de clientes con generación automática de identificadores y validación de fecha de nacimiento, autenticación segura de administradores mediante contraseña, aplicación de polimorfismo para cálculo de descuentos según el nivel de la tarjeta (Básica, Gold, Diamond), herencia en máquinas de arcade (ej. Máquina Garra), y el control en tiempo real de eventos globales y beneficios por cumpleaños.

2. Requerimientos del Sistema

Se conservan los requerimientos funcionales originales (RF01 a RF07) y no funcionales del primer corte, y se añaden nuevos requerimientos funcionales (RF08 a RF13) que corresponden a este corte:

RF08 - Autenticación de Administrador	El sistema debe restringir el acceso al menú de gestión administrativa solicitando una contraseña de seguridad válida.
RF09 - Generación Automática de Identificadores de Tarjeta	El sistema debe generar internamente y de forma automática un código único uncio al registrar la tarjeta vinculada a un cliente
RF10 - Tipo de Tarjetas y Polimorfismo	El sistema debe soportar categorías especializadas de tarjetas(Basica, Gold, Diamond), aplicando de forma polimórfica diferentes límites de saldo máximo y porcentajes de descuento al consumir en las máquinas
RF11 - Validación Automática de Cumpleaños	El sistema debe verificar en tiempo de ejecución la fecha de nacimiento del cliente frente a la fecha del sistema para otorgarle jugadas gratuitas el día de su cumpleaños.
RF12 - Gestión Global de Eventos	El administrador debe poder modificar el estado de los eventos del arcade (como activar el modo <i>Black Friday</i>) para alterar de forma dinámica los costos de las partidas.
RF13 - Especialización de Máquinas	El sistema debe permitir crear diferentes tipos de atracciones mecánicas mediante herencia de una clase padre Maquina (por ejemplo, MaquinaGarra), sobrescribiendo su comportamiento particular de juego.

3. Modelado y Arquitectura del Sistema

3.1 Descripción del Dominio de Clases (Reestructuración total)

- Jerarquía de Usuarios (Herencia):
 - Clase “Usuario”(Padre): Define los atributos comunes(id, nombre, email) y el método general login()
 - Clase “Cliente” (Hija): Hereda de Usuario. Incorpora el atributo privado fechaNacimiento (Date) y el método esSuCumpleanosHoy() para evaluar beneficios temporales.
 - Clase “Administrador” (Hija): Hereda de Usuario. Incorpora el atributo password con validación de seguridad y métodos de gestión global (recargar tarjetas, poner máquinas en mantenimiento, cambiar eventos).
- Jerarquía de Maquinas (Herencia y especialización)
 - Clase “Maquina” (Padre): Administra los datos generales de cualquier atracción (identificador, nombre, costo base de partida y estado activo/mantenimiento).
 - Clases especializadas (ej. MaquinaGarra): Heredan de “Maquina”, sobrescribiendo el método jugar() para adaptar la interacción y entrega de premios particulares.

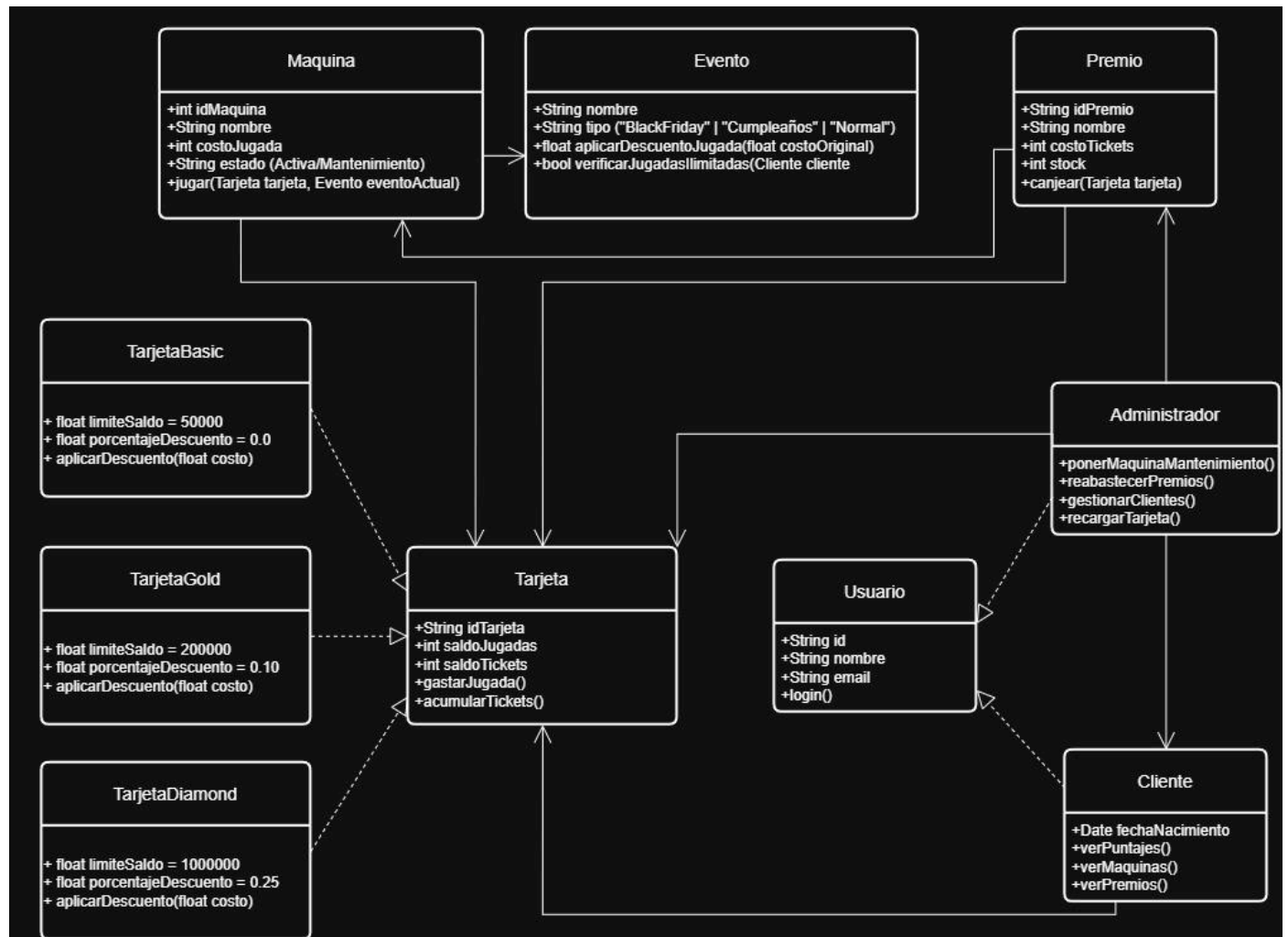
- Clase “Evento”:
 - Responsabilidad: Modela las reglas temporales del centro de entretenimiento (tipo Normal, BlackFriday, cumpleaños), permitiendo que las máquinas recalculen los costos de las partidas de forma dinámica.
- Jerarquía Premios:
 - Clase “Premios”(padre): Administra el nombre y stock del premio en específico.
 - Clase “PremiosPorMaquina”: Toma los atributos de su padre y aparte usa una interfaz para administrar si se entrego el premio o no.
 - Clase “PremiosPorTickets”: Agrega el atributo costoEnTickets para cobrar a la hora de reclamar el premio.

3.2 Relaciones entre Entidades y Diagrama de Clases UML

- Herencia (Generalización): Utilizada para estructurar los roles de usuario (Cliente y Administrador hacía Usuario), los tipos de tarjetas (Basica, Gold, Diamond hacia la clase abstracta Tarjeta) y las máquinas recreativas.
- Asociación y Uso: Se detalla cómo el Cliente posee una Tarjeta, y cómo la Maquina procesa la Tarjeta y consulta el Evento actual en tiempo de ejecución.

3.3 Diagrama de Clases UML

https://drive.google.com/file/d/1VdGuvYPp_ZPeDAAHdGfAGQ3rlrFRAY9t/view?usp=sharing



3.4 Interfaz Gráfica

Para cumplir con el alcance del segundo corte y ofrecer una experiencia interactiva completa, se desarrolló una arquitectura de interfaz gráfica basada en formularios Swing (**javax.swing.JFrame**) en Java. Esta arquitectura usa la lógica de negocio:

Gestión de Navegación y Control de Flujo (ClienteVista y TiendaVista):

- **ClienteVista:** Actúa como el panel principal del usuario final. Permite visualizar en tiempo real los datos del cliente, el saldo de jugadas de la tarjeta, los tickets acumulados y un registro dinámico de los premios redimidos. Agrega botones de acción para navegar hacia la sala de juegos (MaquinaVista), acceder a la tienda de premios (TiendaVista), validar beneficios por cumpleaños y actualizar los datos en tiempo real mediante un enlace directo a los modelos de las tablas (DefaultTableModel).
- **TiendaVista:** Implementa el subsistema de canje de premios. Permite al cliente visualizar su saldo actual de tickets frente a una tabla de premios disponibles (como *Oso de Peluche*, *Consola Mini* o *Bebida Energética*). A través de un componente de selección (JComboBox) y el botón **REDIMIR**, el sistema valida de forma síncrona si el usuario posee los tickets suficientes.

Polimorfismo y Persistencia en Tiempo de Ejecución en la Interfaz:

- Al ejecutar la acción de redimir un premio, la interfaz invoca los métodos definidos en la jerarquía abstracta de Tarjeta (como `getSaldoTickets()` y `setSaldoTickets()`), aplicando los descuentos de tickets de manera segura.
- El nombre del premio redimido se almacena de forma dinámica en una lista interna del objeto Cliente (`listaPremios`), la cual es consultada y renderizada automáticamente en la tabla principal de `ClienteVista` al refrescar la información.

Interconexión de Clases y Vistas:

- Las vistas son las interfaces gráficas que reciben las lógicas del Cliente, la Tarjeta (con sus respectivos tipos: (Básica, Gold, Diamond) y las máquinas: (MáquinaRuleta, MáquinaCarreras, MáquinaDisparos, MáquinaGarra).

¿Cómo se relaciona esto con los Requerimientos del documento?

- **Apoya al Alcance (1.2):** Demuestra cómo el prototipo funcional pasó de consola a una interfaz gráfica interactiva con menús y tablas
- **Refuerza el Modelado (3.1 y 3.2):** Explica visualmente cómo las clases del dominio (Cliente, Tarjeta, Premios) se comunican con los componentes visuales de Java Swing (JTable, JComboBox, JButton) para mostrar los datos procesados en tiempo de ejecución.